AE496 UGP brief: why a physics-informed network loses the cylinder wake, and what brings it back

Peeyush Agarwal October 16, 2026

The question. If you train a physics-informed neural network (PINN) on the flow behind a cylinder without giving it any data, it tends to lose the vortex street and drift towards the steady, symmetric flow. That steady flow satisfies the Navier-Stokes equations and every boundary condition exactly, so the loss has no reason to reject it. Chuang and Barba documented this at $\mathrm{Re} = 200$. I wanted to reproduce the failure on ground truth I had generated and validated myself, and then find out which fix actually defeats it, with controls tight enough that the credit goes to the right mechanism. A second thread follows the reference paper by Renn et al., whose Fourier Neural Operator (FNO) is purely data-driven: does adding physics to an operator forecast help?

Ground truth. I wrote a D2Q9 two-relaxation-time lattice Boltzmann solver in JAX. At $\mathrm{Re} = 100$ it gives $\mathrm{St} = 0.1648$ against Qu et al.'s 0.1649 (within 0.04%), with mean drag 3.5% high and r.m.s. lift 6.2% high. Getting there took two fixes that I did not expect. First, a domain with free-slip side walls is an acoustic duct, and given enough time the wake locked onto its transverse mode (St fell to 0.1132 over 38 periods). A viscous sponge did nothing; a layer that relaxes the flow towards the free stream removed the problem. Second, moving the inlet from 8D to 25D upstream cut the drag error from 7.4% to 2.3% (at $\mathrm{D} = 32$), while the domain height barely mattered. An earlier draft blamed blockage, which was wrong. I then regenerated data at $\mathrm{Re} = 60$, 80, 120, 150 and 180, and every dataset's St is within 0.6% of Qu et al.

The operator. The underlying framework, `underPINN`, only had a steady residual, so I wrote the unsteady Navier-Stokes operator with a faster JVP-based derivative path. It annihilates the Taylor-Green vortex to 1.6×10^{-16}, matches the framework's own steady operator to 5.6×10^{-17}, and runs about 2.5 times faster than the literal Hessian.

The main result. I score each run by *shedding retained*: the r.m.s. transverse velocity at a wake probe, PINN over reference. A plain network trained over one shedding period keeps 31.1% on average, but anywhere from 19% to 43% depending on where in the cycle the run starts. That phase dependence turned out to matter a great deal, so every headline number below is averaged over four start phases with five seeds each. I tested three mechanisms as switches on one trainer: Fourier features (against spectral bias), a Reynolds decomposition around a fitted mean flow, and causal time-marching over four quarter-period windows, where each window starts from the previous window's prediction so the run sees ground truth only once.

configuration (data-free, $\mathrm{Re} = 100$)	retained, phase mean	range over phases	velocity error
plain network, 20 000 epochs	31.1%	19 to 43%	0.166
plain network, 80 000 epochs	48.6%	34 to 63%	0.115
time-marching alone	56.4%	40 to 74%	0.156
all three mechanisms	**72.9%**	**70 to 77%**	**0.031**

The honest reading is more modest than my earlier one. Marching alone gains only 7.7 points over a plain network given the same 80 000 epochs, less than the 17.5 points that longer training adds by itself, and it keeps the phase swing. Fourier features and the decomposition do not improve retention on their own. Put all three together, though, and the result stops depending on start phase, the velocity error drops to a fifth, and marching is worth 34.1 points at equal budget. Over five seeds the phase means carry 95% intervals of 1.5 points or less for the plain and marched networks and 4.6 for all three. The mechanisms are complementary, which agrees with Shah (2026). A little data goes a long way: 100 supervised points per window take retention to 87.4% and 500 points to 96.9%.

Does it forecast? Over one period the best data-free setup tracks the reference phase to 0.1 degrees at 73.5% amplitude. Over five chained periods it keeps shedding, but 10.4% too slowly, and it ends roughly half a cycle out of phase with 63.8% of the amplitude. So it sustains a wake; it does not predict one. Holding the phase takes data, but little: re-anchoring the chain with 350 supervised points per shedding period keeps it within 0.003 of a period of the reference over all five periods, on both seeds, from 1 750 points, about 2% of the 80 000 that a fresh snapshot at every window used.

At 300 points per period it still drifts, so the threshold lies between the two. Across Re = 60 to 180 the pattern holds: all three mechanisms keep 63 to 79% of the wake against 24 to 45% for the plain network, and cut the velocity error from about 0.17 to between 0.028 and 0.040.

Other architectures. A GRU-based sequence PINN keeps less of the wake than the plain network, both on one window (21.8%) and marched (33.6%). A single marched PINNsFormer run reached 77.5%, but at about twelve times the cost, and one run at one phase is a lead rather than a result.

Physics in operators. On the validated data the data-only FNO already forecasts to an error of 0.0010 at its trained horizon, and adding physics makes it 16 to 18% worse; the test states are close to ones it has seen, so this is an easy case. On the earlier, acoustically contaminated dataset physics helps the FNO by 21% at the trained horizon but crosses over within about one period. It leaves a U-Net within 7% and makes a convolutional GRU 30% worse, and both convolutional networks beat the FNO without physics while using 31 and 131 times fewer parameters. Whether physics helps depends on the backbone, not on operator learning in general.

What I got wrong earlier, and fixed. My first headline claimed marching alone kept 89.2% of the wake. Two things inflated it: the marched runs were given a fresh reference snapshot at every window (worth about twenty points), and every configuration was scored at a single start phase. A five-period run that "did not drift" was being re-anchored to the truth at every window, and the ground truth itself was being pulled by the duct mode. Correcting these reversed the conclusion that one mechanism is enough.

Limitations and next steps. The headline has five seeds, but the sweeps, the architecture study and the re-anchoring runs have two; the equal-budget control uses a single budget; the Re sweep uses one start phase per Re; the U-Net and ConvGRU were only run on the earlier dataset; drag stays 3.5 to 5.7% high across the sweep (nothing in the results depends on it). Whether the re-anchoring threshold moves with a longer horizon or another Reynolds number is untested. Enforcing window continuity exactly (Roy and Castonguay) would remove the small kinks at window handoffs. The full PINNsFormer grid would take about 66 GPU-hours.

Verification. Every number in the full report is registered against the run that produced it, and `verify_report.py` currently reproduces all 557 of them from the saved artifacts.